

DEVOIR DE SYNTHÈSE — DEVSECOPS: Sécurisation d'une application Python via une pipeline CI/CD

Contexte général

Dans ce devoir, vous êtes chargés d'appliquer une démarche **DevSecOps complète** sur une application web développée en **Python (Flask)**.

L'application fournie est **fonctionnelle mais volontairement vulnérable**. Elle simule un projet réel développé sans intégrer la sécurité dès la phase de conception.

Votre mission est d'**intégrer la sécurité dans tout le cycle de développement**, à l'aide d'une **pipeline CI/CD automatisée**, en utilisant les outils étudiés en atelier.

Structure du projet

```
devsecops-assignment/  
├── api/  
│   └── app.py  
├── requirements.txt  
├── Dockerfile  
├── .github/  
│   └── workflows/  
│       └── devsecops.yml
```

Code source fourni — **api/app.py**

```
from flask import Flask, request  
import sqlite3  
import pickle  
import subprocess  
import hashlib  
import os  
import logging  
  
app = Flask(__name__)  
  
# SECRET HARDCODÉ (mauvaise pratique)  
API_KEY = "API-KEY-123456"  
  
# Logging non sécurisé  
logging.basicConfig(level=logging.DEBUG)
```

```

@app.route("/auth", methods=["POST"])
def auth():
    username = request.json.get("username")
    password = request.json.get("password")

    # SQL Injection
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = f"SELECT * FROM users WHERE username='{username}' AND
password='{password}'"
    cursor.execute(query)

    if cursor.fetchone():
        return {"status": "authenticated"}
    return {"status": "denied"}

@app.route("/exec", methods=["POST"])
def exec_cmd():
    cmd = request.json.get("cmd")
    # Command Injection
    output = subprocess.check_output(cmd, shell=True)
    return {"output": output.decode()}

@app.route("/deserialize", methods=["POST"])
def deserialize():
    data = request.data
    # Désérialisation dangereuse
    obj = pickle.loads(data)
    return {"object": str(obj)}

@app.route("/encrypt", methods=["POST"])
def encrypt():
    text = request.json.get("text", "")
    # Chiffrement faible
    hashed = hashlib.md5(text.encode()).hexdigest()
    return {"hash": hashed}

@app.route("/file", methods=["POST"])
def read_file():
    filename = request.json.get("filename")
    # Path Traversal
    with open(filename, "r") as f:
        return {"content": f.read()}

```

```

@app.route("/debug", methods=["GET"])
def debug():
    # Divulgateion d'informations sensibles
    return {
        "api_key": API_KEY,
        "env": dict(os.environ),
        "cwd": os.getcwd()
    }

@app.route("/log", methods=["POST"])
def log_data():
    data = request.json
    # Log Injection
    logging.info(f"User input: {data}")
    return {"status": "logged"}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)

```

Fichier requirements.txt

```

flask==1.0
bcrypt==3.1.0

```

Versions volontairement anciennes et vulnérables.

Dockerfile

```

FROM python:3.8

WORKDIR /app
COPY api/ .
RUN pip install flask bcrypt

EXPOSE 5000
CMD ["python", "app.py"]

```

Pipeline CI/CD ([.github/workflows/devsecops.yml](#))

La pipeline doit intégrer **obligatoirement** :

- Analyse SAST du code
- Analyse de sécurité Python
- Scan des dépendances applicatives
- Scan de vulnérabilités de l'image Docker
- Blocage automatique en cas de faille critique

Travail demandé

PARTIE 1 — Mise en place DevSecOps

- Créer un dépôt GitHub
- Intégrer une pipeline CI/CD déclenchée à chaque [push](#)
- Configurer les outils de sécurité étudiés
- Vérifier que la pipeline **échoue** avec le code initial

PARTIE 2 — Analyse des vulnérabilités

À partir des résultats de la pipeline :

- Identifier au minimum 8 vulnérabilités
- Pour chaque vulnérabilité :
 - endpoint concerné
 - type de faille
 - outil de détection

- catégorie OWASP Top 10
- Expliquer le risque associé

PARTIE 3 — Corrections de sécurité

Vous devez corriger l'application en appliquant les bonnes pratiques DevSecOps :

- suppression des injections SQL
- suppression de toute exécution de commandes arbitraires
- suppression de la désérialisation dangereuse
- remplacement du chiffrement faible
- validation stricte des entrées utilisateur
- suppression des secrets hardcodés
- gestion sécurisée des secrets via variables d'environnement
- durcissement du Dockerfile (image allégée, utilisateur non root)

PARTIE 4 — Validation finale

- Re-lancer la pipeline après corrections
- Vérifier que :
 - aucune vulnérabilité **CRITICAL/HIGH** n'est détectée
 - la pipeline devient verte
- Justifier chaque correction apportée

Livrables attendus

- Lien du dépôt GitHub
- Rapport PDF contenant :
 - description de la pipeline

- résultats des outils de sécurité
- tableau des vulnérabilités détectées
- corrections appliquées
- captures d'écran avant / après
- conclusion DevSecOps